

Scalable sub-game solving for imperfect-information games

Huale Li^a, Xuan Wang^{a,b}, Kunchi Li^a, Fengwei Jia^a, Yulin Wu^a, Jiajia Zhang^{a,*},
Shuhan Qi^{a,b,**}

^a School of Computer Science and Technology, Harbin Institute of Technology, ShenZhen, 518055, China

^b Peng cheng Laboratory, 518038, Shenzhen, China

ARTICLE INFO

Article history:

Received 31 August 2020

Received in revised form 7 April 2021

Accepted 21 August 2021

Available online 26 August 2021

Keywords:

Game

Counterfactual regret minimization

Imperfect-information

Agent

ABSTRACT

Counterfactual regret minimization (CFR) is a popular and effective method for solving a game with imperfect information. The effect of CFR is limited by the size of the game state space. With the increase in the number of game participants, the game state space will increase rapidly. Although the vanilla CFR is suitable for two-player imperfect-information games, it does not work well in imperfect-information games with three or more players. In this paper, we design a framework for imperfect-information games, which can not only deal with two-player imperfect-information games but also can efficiently solve three-player imperfect-information games. Compared with traditional solving methods, in this framework we propose real-time hand abstraction (RTHA), which can reduce the error caused by the abstraction. We also propose a warm-start online solution of sub-game (WSOS-SG) method, which can improve the accuracy of the action estimation and solve the sub-game in real time. Experimental results show that the agent based on our method achieve better performances than traditional methods. The agent based on our method took part in the 2018 AAAI-ACPC poker competition and won third place in heads-up no-limit Texas hold'em.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

In recent decades, games have been an important testbed for studying how well artificial intelligence (AI) can perform in complicated decision making. In games with perfect information, players usually can find either an optimal action or an optimal distribution of actions for the current state. In the game of Go, for example, all the information is public for each player, and some search algorithms can be used to seek an optimal action in the current state of the game. By combining a Monte Carlo tree search (MCTS) and deep reinforcement learning, AlphaGo defeated a Go world champion [1–5]. The later improved versions, AlphaGo Zero and Alpha Zero, drew lessons from AlphaGo, which allowed computers to surpass human beings completely in the game of Go [6,7]. However, in many real world applications, such as auctions, negotiations, and stock trades, the information is always imperfect for each game player, which limits the application of perfect-information game methods.

In imperfect-information games, each player aims to find a Nash equilibrium or an approximate Nash equilibrium such that

no single player can improve their utility by deviating from the equilibrium [8]. Currently, to achieve such a goal, counterfactual regret minimization (CFR) and its variants are the most popular algorithms for seeking a Nash equilibrium or an approximate Nash equilibrium in games with imperfect information [9]. By combining the regret minimization algorithm and average strategy calculation method and using an extensive game model to represent the problem of sequential decision making, CFR iteratively converges to a Nash equilibrium or an approximate Nash equilibrium in two-player zero-sum games [10].

In recent years, many improvements based on the vanilla CFR have been proposed. Johanson et al. extended the sampling nodes in the procedure of searching the game tree and proposed three CFR algorithm variants based on chance node sampling [11]. Lanctot et al. designed a general sampling framework for the vanilla CFR and proposed a class of online algorithms based on a Monte Carlo approach (MCCFR), which included CFR based on outcome sampling (OS) and external sampling (ES) [12]. The MCCFR accelerates the speed of the overall convergence by reducing the cost of a single iteration and increasing the number of iterations. As a result, the scale of the problem solved by the MCCFR is about 10 times larger than that of the vanilla CFR. Burch et al. proposed CFR-decomposition (CFR-D) that could handle the trunk and each sub-problem independently [13,14]. Bowling et al. improved the regret value matching algorithm in CFR and proposed the CFR+ algorithm with a faster convergence speed [15]. Brown

* Corresponding author.

** Corresponding author at: School of Computer Science and Technology, Harbin Institute of Technology, ShenZhen, 518055, China.

E-mail addresses: zhangjiajia@hit.edu.cn (J. Zhang), shuhanqi@cs.hitsz.edu.cn (S. Qi).

et al. sped up the convergence of the CFR and proposed a CFR algorithm based on regret value pruning [16]. Jackson proposed a CFR-based method, Compact CFR, for imperfect-information games [17]. Further, Jackson proposed a new sampling variant of the CFR, Targeted CFR [18], which can be viewed as falling somewhere between outcome sampling and external sampling on the spectrum of sampling algorithms. Brown et al. proposed DeepCFR [19], which combined a neural network and vanilla CFR.

Texas hold'em poker is a game with imperfect information, and it is a major event of the ACPC (Annual Computer Poker Competition, held by AAAI and IJCAI in turn) [20,21]. The agents *HITSZ_HKL* [22], *Gibson_3p* [23] and the agent based on neural fictitious self-play (NFSP) [24] are the agents with excellent performance in the ACPC in these years. As a major event of the ACPC, Heads-up no-limit hold'em (HUNL) is also one of the most popular games, which contains 10^{161} states. In the HUNL, each player has private information, and the state space grows as the number of players is increased. However, the vanilla CFR can only solve games with 10^{18} states. The traditional method to solve large-scale games with imperfect information is to abstract the original game before using CFR [25,26]. By using such methods, Deepstack, Libratus, and Pluribus have successfully defeated professional human players [27–30].

Many achievements have been made in the field of imperfect-information games over the past few years, such as the development of Deepstack and Libratus. However, most of these methods focus on games with two players (such as HUNL) [27, 29] and cannot be used in multi-player games (such as three-player no-limit poker), which are extremely common in real-life scenarios [31]. Although Pluribus has recently defeated human professional players in six-player Texas poker [30], determining how to solve the multi-player imperfect-information game is still a challenge problem.

In the paper, we deal with this problem by designing a framework to solve imperfect-information games. The framework consists of real-time hand abstraction (RTHA) and a warm-start online solution of sub-game (WSOS-SG) method. The RTHA is used to abstract player hands in real time during the process of game abstraction. The WSOS-SG can solve the sub-game strategy online in real time during the game. The key idea of the WSOS-SG is that we introduce a warm-start CFR-based method, which can increase the understanding of the situation information and improve the accuracy of the action estimation. Moreover, in the WSOS-SG, we propose a state value estimation method, which increases the estimation precision of the intermediate state value of the sub-game tree.

Noting that our method has strong flexibility, and it can be adapted to two-player to three-player imperfect-information games. In experiments with HUNL and three-player no-limit Texas hold'em poker (TPNL), we showed that the agent based on the proposed method performed much better than some high-level agents. Our agent based on the proposed method participated in the 2018-ACPC and finished in the third place. We summarize our contributions as follows:

- (1) We designed a framework based on abstraction and CFR, which was used to study the solutions of imperfect-information games with two and three players.
- (2) In the designed framework, RTHA and WSOS-SG based on the CFR are presented to solve the strategy of the imperfect-information game, which reduce the error caused by abstraction and improve the accuracy of the final solution strategy.
- (3) Experimental results showed that the agent based on our method achieved better performances compared to traditional methods.

The rest of our paper is organized as follows. In Section 2, we introduce the concepts of the extensive-form game, describe

the Nash equilibrium and CFR algorithm, and discuss the strictly dominant strategy. Our method is presented in Section 3, the framework is introduced in Section 3.1, and the RTHA in the game abstraction is introduced in Section 3.2. Furthermore, the strategy solving method WSOS-SG is discussed in Section 3.3. In Section 4, we evaluate the performance of our method in THUL and TPNL. Finally, in Section 5, the content of the paper is summarized, and future work is discussed.

2. Background

We first introduce the notation and definitions of extensive-form games. We then introduce the details of the Nash equilibrium. After this, we discuss the popular CFR algorithm. Finally, we conclude this section by discussing the concepts of the strictly dominant strategy. Since many variables will be used in this paper, a detailed description of the variables is given in Table 1.

2.1. Extensive-form game

The extensive-form game is a classical model for an imperfect-information game. A game tree can be used to represent the extensive-form game. The unique initial state of the game is represented by the root of the game tree. Choice nodes are possible decision points of each player. The edges leaving a choice node represent the legal actions that can be taken by the corresponding player. Terminal nodes represent the end of the game and contain the utilities of all players reaching that point [32,33]. Fig. 1 shows a game tree of the game rock-paper-scissors. As shown in Fig. 1, player 1 takes the action 'rock,' player 2 also takes the action 'rock,' and then the utility of both are zero. This is the full process of the game.

There are often several states that a player cannot differentiate, since opponents have private information (such as their private cards in poker) in imperfect-information games. Here information sets are these sets of all indistinguishable game states for that player. Each choice node corresponds to an information set (instead of including a node for every game state) in an extensive-form game with imperfect information. In poker, there are many more game states than information sets.

Definition 1. A finite extensive-form game Γ with imperfect information has the following six components $\langle N, H, P, f_c, u, I \rangle$:

- (1) A finite set N of players, $N = 1, 2, \dots, n$;
- (2) A finite set H of sequences of possible action histories. The empty sequence and every prefix of a sequence are also in H . $Z \subseteq H$ are the terminal histories, which are not a prefix of any other sequences. $A(h) = \{a \mid (h, a) \in H\}$ are the actions available after a non-terminal history $h \in H$.
- (3) A function P that assigns to each non-terminal history (each member of $H \setminus Z$) a member of $N \cup \{c\}$, where P is the player function. $P(h)$ is the player who will take an action after the history h . If $P(h) = c$, then chance determines the action after the history h .
- (4) A function f_c associated with every history h for which $P(h) = c$ is a probability measure $f_c(\cdot|h)$ on $A(h)$ ($f_c(\cdot|h)$ is the probability that a occurs given h), where each such probability measure is independent of every other such measure.
- (5) For each player $i \in N$, a partition I_i of $\{h \in H : P(h) = i\}$ with the property that $A(h) = A(h')$ whenever h and h' are in the same member of the partition.
- (6) For each player $i \in N$, a utility function u_i from the terminal states Z to the reals R . If $N = 1, 2$ and $u_1 = -u_2$, it is a zero-sum extensive game.

Table 1
Detailed descriptions of the variables used in the paper.

Variable	Detailed description
H	A finite set of sequences, possible histories of actions. h is a history (i.e., node), $h \in H$. $Z \subseteq H$ are the terminal histories.
I	The information set in imperfect-information games. For any information set I_i belonging to player i , all nodes $h, h' \in I_i$ are indistinguishable to the player i .
$A(h)$	The actions available at a node h .
$p(h)$	The player whose turn it is to act at the node h . If chance acts the node, then $P(h) = c$.
u_i	A utility function u_i from the terminal states Z to the reals R . If $N = 1, 2$; and $u_1 = u_2$, it is a zero-sum game.
σ_i	A probability vector over actions for the player i in the information set I . A strategy profile σ is a tuple of strategies, one for each player. σ_{-i} is the strategy of all players other than the player i .
$\pi^\sigma(h)$	The probability with which h is reached if all players play according to the strategy σ . $\pi_i^\sigma(h)$ is the contribution of i to this probability. $\pi_{-i}^\sigma(h)$ is the contribution of chance and all players other than i .
$R_i^T(I, a)$	Average overall regret in the CFR, defined in Eq. (4).
t	The number of the iteration in the CFR, T is the total iteration number.
$v_i(\sigma, I)$	The counterfactual value of an information set I where $P(I) = i$ is the expected utility to player i given that I has been reached, weighed by the external reach of I for player i , as shown in Eq. (5).

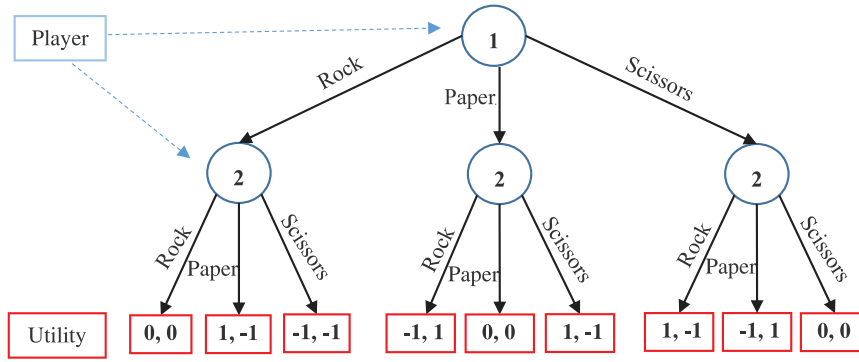


Fig. 1. Game tree of the game rock-paper-scissors (circles represent the players of the game, boxes represent the benefits after the game, and the connecting lines between the circles represent the legal actions allowed by the game.)

Definition 2. A strategy σ_i of player i in an extensive-form game is a function that assigns a distribution over $A(I_i)$ to each I_i , and Σ_i is the set of strategies for the player i . A strategy profile σ consists of a strategy for each player, $\sigma_1, \sigma_2, \dots, \sigma_n$, where σ_{-i} refers to all the strategies in σ except σ_i .

Let $\pi^\sigma(h)$ be the probability of history h occurring if players choose actions according to σ . We can decompose $\pi_i^\sigma(h) \prod_{i \in N \cup \{c\}} \pi_i^\sigma(h)$ into each player's contribution to this probability. Hence, $\pi_i^\sigma(h)$ is the probability that if the player i plays according to σ , then for all histories h' that are a proper prefix of h with $P(h') = i$, the player i takes the corresponding action in h . Let $\pi_{-i}^\sigma(h)$ be the product of all players' contributions (including chance) except the player i . For $I \subseteq H$, define $\pi^\sigma(I) = \sum_{h \in I} \pi^\sigma(h)$, as the probability of reaching a particular information set given σ , with $\pi_i^\sigma(I)$ and $\pi_{-i}^\sigma(I)$ defined similarly.

The overall value to the player i of a strategy profile is the expected payoff of the resulting terminal node, $u_i(\sigma) = \sum_{h \in Z} u_i(h) \pi^\sigma(h)$.

2.2. Nash Equilibrium

Nash equilibrium and ε -Nash equilibrium are often used in two-player zero-sum games [8]. A Nash equilibrium is a strategy profile σ , in which no player can increase their utility by unilaterally changing their strategy. That is, if each player is provided with the static strategies of all the other players and still cannot gain any utility by changing their strategy, then they are all playing at a Nash equilibrium. We introduce the best response to present Nash equilibrium more clearly.

Definition 3. A best response b_i is a strategy that obtains the highest utility against the set of all other strategies in a strategy

profile σ :

$$b_i(\sigma_{-i}) = \max_{\sigma'_i \in \Sigma_i} u_i(\sigma'_i, \sigma_{-i}), \quad (1)$$

where Σ_i represents all possible strategy profiles for the players and u_i represents the benefits they obtain.

In poker, this corresponds to the strategy that wins the most chips against all other players, given that their entire strategies have been revealed and remain static (they do not change). A best response to an abstract game is confined to playing in the abstract game (as opposed to the original game).

Notice that each player in a Nash equilibrium is a best response to the set of all other players. More formally, a strategy profile for n players, $\sigma_1, \sigma_2, \dots, \sigma_n$, is a Nash equilibrium if it satisfies the following constraints:

$$u_i(\sigma) \geq \max_{\sigma'_i \in \Sigma_i} u_i(\sigma'_i, \sigma_{-i}), \quad (2)$$

where σ_i^* is a Nash equilibrium for the player i . Thus, we see that, in a Nash equilibrium, there is no strategy for player i in Σ_i that yields a higher utility against σ_{-i} than its strategy, σ_i , in σ . In fact, in a two-player zero-sum game, each player who plays a Nash equilibrium strategy has the same value regardless of which Nash equilibrium strategy each player plays.

Computing an exact Nash equilibrium for a large game such as poker (or even an abstraction) is infeasible. Thus, we are often interested in finding approximations to Nash equilibria, the ε -Nash equilibrium, such that

$$u_i(\sigma) + \varepsilon \geq \max_{\sigma'_i \in \Sigma_i} u_i(\sigma'_i, \sigma_{-i}). \quad (3)$$

An ε -Nash equilibrium is a strategy profile, σ , in which no player can increase their utility more than ε by unilaterally changing their strategy.

2.3. Counterfactual regret minimization (CFR)

CFR has been a popular algorithm for finding Nash equilibria in large imperfect-information games over the past several years [9]. CFR is an iterative algorithm that minimizes a form of regret, positive immediate counterfactual regret, for each information set. As the number of iterations increases, the average overall regret decreases, and the iteration strategy will approach a Nash equilibrium. Average overall regret is defined as follows:

$$R_i^T(I, a) = \frac{1}{T} \sum_{t=1}^T (v_i(I, \sigma_{I \rightarrow a}^t) - v_i(I, \sigma^t)), \quad (4)$$

where T is the number of games played, and $\sigma_{I \rightarrow a}^t$ is an equivalent strategy profile of σ , except that player i chooses the action a in the information set I in $\sigma_{I \rightarrow a}^t$. v_i is the counterfactual value, defined as follows:

$$v_i(\sigma, I) = \sum_{z \in Z_I} u_i(z) \pi_{-i}^\sigma(z[I]) \pi^\sigma(z[I], z), \quad (5)$$

where Z_I is the action sequence of the information set I , $z[I]$ is the action sequence from the root node to the information set I , $u_i(z)$ is the utility of player i obtained in the leaf node z , $\pi_{-i}^\sigma(z[I])$ is the probability that the opponent reaches the information set I , and $\pi^\sigma(z[I], z)$ is the product of the probability for all players from the information set I to the leaf node z .

The next iteration strategy in the information set I can be represented as follows:

$$\sigma_i^{T+1}(I, a) = \begin{cases} \frac{R_i^{T,+}(T, a)}{\sum_{a \in A(I)} R_i^{T,+}(T, a)}, & \sum_{a \in A(I)} R_i^{T,+}(T, a) > 0 \\ \frac{1}{|A(I)|}, & \sum_{a \in A(I)} R_i^{T,+}(T, a) \leq 0 \end{cases} \quad (6)$$

The average strategy of the players computed with the CFR will eventually converge to Nash equilibrium through a large number of iterations, and the average strategy is as follows:

$$\bar{\sigma}_i^T(I, a) = \frac{\sum_{t=1}^T \pi_i^{\sigma^t}(I) \sigma^t(I, a)}{\sum_{t=1}^T \pi_i^{\sigma^t}(I)}. \quad (7)$$

Since the CFR was proposed, many variants have appeared with improved computational speeds, game tree sizes, and sampling methods, such as CFR+, Linear CFR, MCCFR and CFR-D [11–13,15,34,35].

2.4. Strictly dominant strategy

Since the CFR has no theoretical guarantee for multiplayer games, the strictly dominant strategy is introduced to solve the sub-optimal strategy in multiplayer games by using the CFR. The CFR can remove a strictly dominant strategy after a large number of strategy iterations, and Gibson [36] verified the feasibility of the CFR in solving multi-player games with imperfect information through experiments. The definition of the dominant action and the dominant strategy are as follows:

Definition 4. The action of player i is $a \in A(I)$ in an extensive-form game. If exists another strategy $\sigma'_i \in \Sigma_i$ for player i , any strategy profile $\sigma_{-i} \in \Sigma_{-i}$ for other players there are $v_i(I, \sigma_{I \rightarrow a}) \leq v_i(I, (\sigma'_i, \sigma_{-i}))$, the action a is a weak dominant action. In this case, the dominant action can be defined recursively as an action that is dominant or becomes dominant after eliminating the strictly dominant action in the process of the strategy iteration.

Definition 5. A strategy σ_i of player i is a weak dominant strategy if there exists another strategy σ'_i that satisfies the following conditions:

- (1) For any strategy $\sigma_{-i} \in \Sigma_{-i}$ for the other players, $u_i(\sigma_i, \sigma_{-i}) \leq u_i(\sigma'_i, \sigma_{-i})$;
- (2) For some strategies $\sigma_{-i} \in \Sigma_{-i}$ for the other players, $u_i(\sigma_i, \sigma_{-i}) < u_i(\sigma'_i, \sigma_{-i})$.

If all strategies $\sigma_{-i} \in \Sigma_{-i}$ for the other players satisfy $u_i(\sigma_i, \sigma_{-i}) < u_i(\sigma'_i, \sigma_{-i})$, then σ_i is a strictly dominant strategy. The dominant strategy can be defined recursively as a strategy that is dominant or becomes dominant after eliminating the strictly dominant strategy in the strategy iteration process.

3. Our method

In this section, we present the details of our proposed method. First, we introduce the framework for solving an imperfect-information game. We then introduce the RTHA for game abstraction. Finally, we discuss the details of the proposed strategy solving method WSOS-SG.

Most of existing methods that use the CFR to seek a Nash equilibrium aim at solving two-player imperfect-information games. In this paper, to solve multi-player imperfect-information games, we design a framework to solve the game strategy. In the framework, we propose the RTHA to abstract player hands in real time. We also propose a warm-start based method for sub-game online solution solving (WSOS-SG), which consists of three parts: a real-time hand distribution prediction (RTHD), intermediate state value estimation in sub-game (ISVE-SG) method, and a warm-start solution of sub-game method based on CFR (WS-SG).

3.1. Framework overview

We designed a framework to solve the strategy of the imperfect-information game. Our framework can solve the strategy of imperfect-information games with two and three players effectively. Moreover, the framework can be easily extended to imperfect-information games with over three players. The framework is shown in Fig. 2.

The framework of the proposed strategy solving process mainly consists of two parts: game abstraction and strategy solving. We propose the RTHA to abstract player hands in real time in the game abstraction process, and the WSOS-SG process to solve the strategy in the game, which includes RTHD, ISVE-SG, and WS-SG.

3.2. Game abstraction

The imperfect-information game contains private information, and the state space becomes much larger as the number of the player increases. Thus, the strategy solution of an imperfect-information game is more complicated than that of an perfect-information game. To solve this problem, we propose the RTHA method, which can abstract the game state in real time.

Real-Time Hand Abstraction (RTHA) Since the number of possible combinations of private and public cards decreases as the game progresses, the state space of the game tree can be greatly reduced by removing the impossible combinations of hands from the issued public cards, and then the hands are abstracted. Table 2 shows the number of all combinations and the number of combinations after removing some impossible combinations in three rounds. Thus, real-time hand abstraction can greatly reduce the state space of the game and improve the accuracy of hand abstraction as the game progresses.

The RTHA mainly includes the following two steps:

Step 1: Calculate the strength values of various combinations of the hands abstracted by the hand isomorphism algorithm through offline abstraction [37], which can accelerate the speed of the calculation when using hand abstraction in real time.

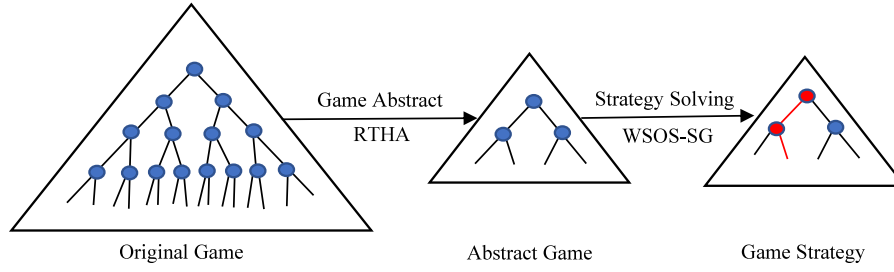


Fig. 2. Framework of strategy solving. Since the original game has a larger state space than the abstract game, the triangle of the original game is larger than the other two triangles.

Table 2
Number of combinations in three rounds.

Round	Number of original combinations	Number of combinations after removing public cards
Flop	28094800000	1271256
Turn	14047000000	1128
River	2809476000	1081

Step 2: When abstracting the hands in real time, different kinds of hands are abstracted based on the position of the current game situation in the original game tree. First, if the current situation of the game is in the pre-flop round, the offline abstraction by hand isomorphism from step 1 is used directly. Meanwhile, the total number of combinations of the hands is $C(52,2) = 1326$, and the number of combinations of hands after using hand isomorphism is 169. Second, if the game is currently in the flop, turn, or river round, the algorithm removes the public cards that have been issued. Third, the algorithm looks up the strength of the possible combinations of hands calculated in step 1. Finally, the k-means algorithm [38] is used for clustering the hands.

3.3. Strategy solving with WSOS-SG

After the state space of the game is compressed by the RTHA, the method of strategy solving of the WSOS-SG for imperfect information is used in the process of solving the game.

3.3.1. Real-time hand distribution prediction (RTHD)

As the game progresses, players expose more and more information to their opponents, because the actions they have taken are related to the player's hand strength. Here the strength of the hand can be expressed as a probability. For example, Texas hold'em poker has 52 cards, and the hands of opponents can be represented by the remaining 50 cards in the preflop round. In this case, our hand is better(+1), tied(0) and worse(-1) compared with the hand strength of opponents. The hand strength can be computed through taking the summation and divided by the total number of possible opponents' hands. If a player takes the action of raising chips in the pre-flop and flop rounds, this player's hand strength may be strong; the more chips the player raises, the greater the strength of the hand the player may have. Furthermore, if the player only takes the action of calling, the strength of the player's hand may be weak. The behavior of the same player is always stable. In view of this, we analyze the player's play style according to their game data history (such as betting actions). In this way, we predict the player's private hands more accurately according to the player's actions, which is very similar to human behavior of reading hands in playing games.

The establishment of the specific model of a player is needed, when we predict the player's hand distribution according to the action sequence taken by the player. However, it cannot be effectively established in the absence of the history of the game data

Algorithm 1 Algorithm for RTHD

Input: Active players $P = P_1, \dots, P_n$, public cards B , number of private cards H , current betting history h , the original hand distribution D of active players, the trunk strategy σ or a strategy σ_i of the sub-game, and the last round action sequence As .

Output: Normalized hand distributions D .

```

1: for player  $i \in P$  do
2:    $D'_i \leftarrow [0]_H$ 
3:   for  $P_1 = 0$  to 50,  $P_1$  is not on  $B$  do
4:     for  $P_2 = 1$  to 51,  $P_2$  is not on  $B$  do
5:        $I \leftarrow \text{IndexHoles}(P_1, P_2)$ ; //build hand index of each
       player
6:        $p_i \leftarrow$  the probability of the player  $i$  takes action
        $a_i \in As$  according to the strategy  $\sigma$  or  $\sigma_i$ ;
7:        $D'_i[I] \leftarrow p_i$ ;
8:       end for
9:     end for
10:     $D_i \leftarrow D_i \odot D'_i$ ; //multiply the corresponding element
11:     $\text{Normalize}(D_i)$ ; // normalize the hand distribution of each
       player
12:  end for
13: return  $D$ .
```

for a specific player. In this paper, we assume that the opponent's strategy is the real-time strategy we calculated, and then Bayes' formula [39] is used to calculate the player's hand distribution according to the action taken by the player.

Assuming that σ is the real-time strategy computed by the agent, all the possible hands of the player i are $C_1, C_2, \dots, C_n$. Action A will be taken according to the strategy σ . The conditional probability of the player's hand after taking action A can be calculated as follows:

$$P(C_i) = P(C_i | A) = \frac{P(A | C_i) \times P(C_i)}{\sum P(A | C_j) \times P(C_j)}, \quad (8)$$

where $P(A | C_j)$ is the probability of taking the action A when C_j is the hand of the player i . The normalization of $P(C_i)$ is the distribution of the hand after the player i takes the action A . The whole process is depicted in Algorithm 1.

3.3.2. Intermediate state value estimation in sub-game (ISVE-SG)

When calculating the strategy of the current game, the size of the sub-game tree is determined by the position of the current situation in the original game tree. If leaf nodes of the sub-game tree do not reach the end of the game, it is necessary to estimate the value of these intermediate leaf nodes. There are many methods to estimate the payoff of leaf nodes in the sub-game tree, such as MCTS, and deep neural networks [2,40].

Since many existing value estimation methods of leaf nodes are based on the comparison of the current players' hand

Algorithm 2 Algorithm for ISVE-SG

Input: Active players N , current game state S , the hand distribution of players D , the action sequence history h , the hand abstraction C , the strategy of players in the last iteration σ , and the value of the leaf node v .

Output: Value of the leaf node v .

Establish a sub-game tree with the current leaf node as the root node.

```

1: Initialize:  $CurrentNode \leftarrow S$ ;  $v \leftarrow []$ ;
2: if  $CurrentNode \in T$ , the game is end in this round then
3:   Renew  $v$  by comparing the hand strengths of players;
4: else
5:   Renew  $v$  with UCT or MCTS;
6: end if
7: return  $v$ .
```

strengths. There are some problems in calculating the values of the leaf nodes through these methods. First, the exploitability of the calculated strategy of the game is relatively high (e.g., the agent does not consider some of the opponents' skills and is more direct in choosing actions). Second, it needs to assume that the opponent has adopted the strategy we calculated before, and if the calculated strategy is bad, then the error of the prediction will be large. As the game progresses, errors will gradually increase.

To solve the above problems, we propose ISVE-SG method. In the ISVE-SG, depending on whether the solution of the sub-game tree can reach the leaf nodes, we divide the estimation of the intermediate state into two cases:

Case 1: A sub-game tree is established with the current leaf node as the root node, and the leaf node of the sub-game tree only reaches the end of the current round. If the leaf node of the sub-game tree is at the end of the game, the player's utility is estimated by the hand strength.

Case 2: If the leaf node of the sub-game tree is not at the end of the game, the algorithm of upper confidence bound apply to tree (UCT) [41] or MCTS [2] is used to calculate the value of the current leaf node.

Through the above methods, we deal with the different arrival modes of the leaf nodes in the sub-game with finer granularity to improve the accuracy of the intermediate state value estimation. The ISVE-SG in our paper is shown in Algorithm 2.

3.3.3. Warm start online solution of sub-game based on CFR (WS-SG)

We proposed the WS-SG, which can not only solve the problem of a two-player game with imperfect information but can also effectively solve the strategy of a three-player game with imperfect information. Due to the huge state space of imperfect-information games, the loss of information is significant by using the game abstraction, and the scale of the state space increases exponentially with the increase in the number of players. Thus, it is unreasonable to consider the game as a whole to solve the strategy.

Compared with the traditional method of solving game strategy based on state abstraction, the WS-SG no longer regards the game tree as a whole but establishes an abstract sub-game tree through state abstraction for each current game. Our method estimates the value at the leaf node of the sub-game tree and then uses the CFR or MCTS algorithm to calculate the strategy of the current game in real time. With the progress of the game, more and more actions are taken by players, and public cards are issued gradually. Based on the actions taken by players and the public cards issued, the distribution of opponents' hands is predicted in real time. Meanwhile, the chips of the current pot are used as the input of the agent to correct the estimation of the

pot by the agent. When solving the strategy of the current game, the distribution of each player's hand can be predicted by using the actions taken by each player before and the public cards are issued.

Bayes' formula [39] is used to calculate the current distribution of each player's hand according to the strategy computed in the previous step, and agents can further reduce the size of the sub-game tree through RTHA based on the issued public cards.

The proposed WS-SG method is shown in Algorithm 3. The WS-SG mainly includes two parts: the trunk strategy (computed offline, only used in the pre-flop round), and the online strategy. The WS-SG is completed according to the following steps:

Step 1: Abstract the hand in real time with the hand abstraction algorithm and compute a trunk strategy;

Step 2: Judge whether it is the turn of the agent to take action:

a. If it is, renew the distribution of the hand of the player, establish a sub-tree of the game, compute the real-time strategy of the current state of the game, and take action based on the real-time strategy;

b. If it is not, renew the strategy of the players, take action according to the previous action sequence, renew the distribution of the hands of the players, and update the real-time strategies of the players;

Step 3: The game continues until the end, and the next loop begins.

Algorithm 3 Algorithm for WS-SG

Input: Active players N , current game state S , original game tree T , the hand distribution of players $D = D_1, \dots, D_N$, the action sequence history h , the hand abstraction C , the strategy of players in the last iteration σ , and the trunk strategy σ^t .

Follow the trunk strategy σ^t until a leaf node or some mid-game is reached.

Establish a sub-game tree with the current leaf node as the root node.

```

1: Initialize:  $CurrentNode \leftarrow S$ ;  $As \leftarrow []$ ; //traverse from current state
2: if  $CurrentNode \in T$  and we are active then
3:   Renew  $D$  with trunk strategy  $\sigma^t$ , and betting history  $h$ ;
4: end if
5: while  $CurrentNode \in T$  and the game is not end do
6:   if  $CurrentNode \in$  beginning of betting round then
7:     Remove the combination with the public cards, use the RTHA to abstract the hand, normalize the probability of the hand distribution;
8:     if  $As \neq []$  then
9:       Renew  $D$  with  $As$ ,  $\sigma$ , and the betting history  $h$ ;
10:       $As \leftarrow []$ ;
11:    end if
12:   else if  $CurrentNode \notin$  beginning of betting round then
13:     if  $As \neq []$  then
14:       Renew  $D$  with  $As$ ,  $\sigma$ , and the betting history  $h$ ;
15:       $As \leftarrow []$ ;
16:    end if
17:     Establish a sub-game tree, use CFR to update strategy  $\sigma$ , and wait for the active player to take action  $a$ ;
18:      $S \leftarrow ChildState(S, a)$ ; //assign successor node to the current node
19:      $As.append(a)$ ;
20:   end if
21: end while
```

Table 3
Details of the two kinds of poker.

Game	Player	Round	Total cards	Public cards	Private cards	Big blind
HUNL	2	4	52	5	2	100
TPNL	3	4	52	5	2	100

4. Experiments

In this section, we quantitatively evaluate the performance of the agents based on our method and compare with existing agents in the HUNL and the TPNL.

4.1. Experiment setup

To evaluate the proposed method, we selected HUNL and TPNL poker as the research objects. The experiments were conducted based on the rules of the ACPC, which is held by the AAAI/IJCAI every year. The ACPC specifies that each agent's total chip value is updated to 20000, the small blind is 50 chips, and the big blind is 100 chips when the game begins.

The HUNL is a two-player poker game. It is a repeated game, in which the two players play a match of individual games, usually called hands, while alternating who is the dealer. In each of the individual games, one player will win some number of chips from the other player, and the goal is to win as many chips as possible over the course of the match. Each individual game begins with both players placing a number of chips in the pot: the player in the dealer position puts in the small blind, and the other player puts in the big blind, which is twice the small blind amount. During a game, a player can only wager and win up to a fixed amount known as their stack, in the format of HUNL used in the ACPC and this paper, the big blind is 100 chips and the stack is 20000 chips or 200 big blinds.

HUNL and TPNL both consist of four rounds: pre-flop, flop, turn, and river. There are 52 cards in total. In the pre-flop round, each player is dealt two cards by the dealer. These two cards are the player's private hands and are unobserved by their opponents. In the flop round, the dealer deals three public cards. A card is dealt both in the turn and the river. Finally, there are a total of five public cards over four rounds on the board. Each player can form the strongest five-card hand to compare with other players. Each player can use any cards from their two private cards and five public cards to form their hand.

After cards for the round are dealt, players alternate taking actions of three types: fold, call, or raise. Folding means giving up on the current game and forfeiting all wagers placed in the pot. Calling means the player places a number of chips in the pot to equal to the opponent's share. Raising means the player adds more chips to the pot than the opponent's share. Players cannot raise beyond the 20000 chips they start with under the ACPC rules.

Experiments were performed following the ACPC rules. The agents in the experiments applied 32 threads to accelerate the computation of the real-time strategy. The RTHA was used to decrease the number of the game states for all the experiments. During the pre-flop, the abstraction buckets the hand into 169 abstract hands. During the other three rounds, the abstraction buckets the hand into 200 abstract hands. Because no-limit poker has a huge action space (the number of bet chips is continuous if the chips are available), we adopt action abstraction [25]. In our experiments, the game included five bet sizes for every round of the game (0.5, 1.0, 2.0, and 5.0 times the size of the current pot and all in). A player can also decide not to bet. Table 3 provides more details of the two types of poker.

Evaluation criterion: The performance of the agent is measured in terms of milli big blinds per game (mbb/g), which is a standard measure of the win rate in poker games. Researchers have standardized on the unit milli-big-blinds per game, or mbb/g, where one milli-big-blind is one thousandth of one big blind.

4.2. Result comparison

In this section, the proposed methods are tested on HUNL and TPNL poker. To objectively test the performance of our method and the effectiveness of each improved module, we selected two strong agents for comparison, *HITSZ_HKL* and *Gibson_3p* [22,23]. *HITSZ_HKL* [22] is the agent that took the third place in HUNL in the 2017-ACPC, which uses the traditional state abstraction method and then calculates the approximate Nash equilibrium with the Pure-CFR algorithm to obtain the offline strategy. Card abstraction is used in the *HITSZ_HKL* agent. The abstraction buckets the hand into 169, 300, 1000, and 1000 abstracts hands in turn in the pre-flop, flop, turn, and river, respectively. The *HITSZ_HKL* includes five bet sizes at every round in the game (0.75, 1.0, 2.0, and 3.0 times the size of the current pot and all in). Another agent, *Gibson_3p* [23], was applied as the opponent agent for TPNL poker, which does not use card abstraction. It includes three bet sizes in every round (0.5 and 1.0 times the size of the current pot and all in).

We conducted four groups of experiments to verify our method. We designed four agents (agent1, agent2, agent3, and agent4) in the experiments. The performances of our agents were gradually enhanced to test the effectiveness of each module. In other words, the experimental results of our final agent are shown in Table 7. In addition, for agent4 implemented with our method, we also selected six high-level agents reported in recent years to carry out comparative experiments. The experimental results are shown in Table 8.

All four agents used the RTHA and RTHD to abstract the state of the game and renew the hand distribution, respectively. When solving the strategy, agent1 used UCT and agent2 used CFR. Agent3 used RTHD and CFR to solve the strategy. Agent4 used the WSOS-SG to compute the strategy in the games. All four agents could be used in both HUNL and TPNL poker. Table 4 shows the agents and their methods.

4.2.1. Performance of the online and offline strategy

Our first experiment compared the performance of the online and offline strategies. For strategy solving, we used the CFR and UCT algorithms. Table 5 shows the performance of each technique. In all our experiments, the results were measured in the standard units used in this field: milli-big-blinds per hand (mbb/h).

As shown in Table 5, agent1 and agent2 based on the online strategy were worse than the agents based on the state abstraction and the offline strategy, both losing more than 500 mbb/h over 20000 hands. This was because the online strategy received very little information at the beginning of the game. Meanwhile, the error of the strategy using real-time computing directly was relatively large. Moreover, in the first round of poker, public cards were not issued, and the scale of the game state and the value estimation error of the intermediate state were relatively large, which resulted in the non-ideal performance of the agent.

In addition, the experimental results in Table 5 showed that agent1 based on the UCT was worse than agent2 based on CFR. This was mainly because the UCT required a certain number of searches, but the real-time calculation was limited by the time and the computer hardware, which resulted in an insufficient number of searches. Thus, in later experiments, we used the CFR to solve the strategy if there was no special explanation.

Table 4

Agents considered in this paper and their methods.

Agent	Method
HITSZ_HKL	Traditional state abstraction method, uses Pure-CFR to solve strategy, only for two-player games, in [22]
Gibson_3p	Uses CFR to solve strict dominated strategy, only for three-player games, in [23]
Agent1	Uses RTHA to abstract the original game, uses RTHD to renew hand distribution, uses UCT to solve strategy
Agent2	Uses RTHA to abstract the original game, uses RTHD to renew hand distribution, uses CFR to solve strategy
Agent3	Uses RTHA to abstract the original game, uses RTHD to renew hand distribution, uses ISVE-SG and CFR to solve strategy
Agent4	Uses RTHA to abstract the original game, uses RTHD to renew hand distribution, uses WSOS-SG to solve strategy
NFSP	Combines deep reinforcement learning and deep learning, details provided elsewhere [24]
DeepStack1	Uses deep learning, self-play values within the continual re-solving computation, details provided elsewhere [27]
Target CFR	Uses target CFR, details provided elsewhere [18]
CFR+	Uses CFR+, details provided elsewhere [42]
Compact CFR	Uses compact CFR, details provided elsewhere [17]
Deep CFR	Uses Deep CFR, details provided elsewhere [19]

Table 5

Performance of various strategy-solving methods in two games.

Poker	Agent	Hand			
		6000	9000	12000	20000
HUNL	Agent1 vs. HITSZ_HKL	−1150.7	−1260.3	−1397.1	−1337.4
	Agent2 vs. HITSZ_HKL	−950.7	−1060.3	−997.1	−937.4
TPNL	Agent1 vs. Gibson_3p	−550.7	−664.3	−697.1	−537.4
	Agent2 vs. Gibson_3p	−450.3	−563.1	−574.8	−517.2

Table 6

Performance of the poker agent with the ISVE-SG.

Poker	Agent	Hand			
		6000	9000	12000	20000
HUNL	Agent2 vs. HITSZ_HKL	−950.7	−1060.3	−997.1	−937.4
	Agent3 vs. HITSZ_HKL	−350.7	−254.3	−257.1	−217.4
TPNL	Agent2 vs. Gibson_3p	−450.3	−563.1	−574.8	−517.2
	Agent3 vs. Gibson_3p	−150.7	−154.3	−157.1	97.4

Table 7

Performance of the poker agent with the WSOS-SG.

Poker	Agent	Hand			
		6000	9000	12000	20000
HUNL	Agent2 vs. HITSZ_HKL	−950.7	−1060.3	−997.1	−937.4
	Agent3 vs. HITSZ_HKL	−350.7	−254.3	−257.1	−217.4
	Agent4 vs. HITSZ_HKL	70.4	91.7	35.9	37.4
TPNL	Agent2 vs. Gibson_3p	−450.3	−563.1	−574.8	−517.2
	Agent3 vs. Gibson_3p	−150.7	−154.3	−157.1	97.4
	Agent4 vs. Gibson_3p	250.7	354.3	257.1	197.4

4.2.2. Component analysis of framework

Our second group of experiments tested the effect of using ISVE-SG in games. We concluded that the estimation errors of the intermediate state of the game affected the accuracy of the strategy solving through the first experiment. In the traditional algorithm, the sub-game tree was restricted within the current rounds. Victory or defeat was judged directly based on the hand strength of each player in the leaf node. That is, the game issued all the public cards at the current leaf node, and then each player's hand strength was compared to decide whether they won or lost. The game directly ended at the current node, and it did not enter the next round. As a result, the agent tended to take the “all in” action or raise more chips when the hand strength was stronger, while the action of calling or raising smaller chips from the opponent would be interpreted as the hand strength of the opponent being weak. Table 6 shows the performance of agent3 with the ISVE-SG.

Table 6 shows that agent3 based on the ISVE-SG was still worse than the competition agent, but the performance of agent3 was significantly better than that of the previous agents in the first group experiments. Although these agents based on traditional methods through offline calculations were better than our

agent under the ACPC rules, the hand and the action were fixed after abstraction, which greatly reduced the practicability. Our method was implemented in real time and could flexibly adjust to the actual situation.

4.2.3. Performance of agent4 based on WSOS-SG

Our third group of experiments tested the performance of agent4 based on the WSOS-SG. Through the second experiment, we found that although the performance of agent3 had increased significantly compared with the agents in the first experiment, it was still worse than the agent based on the traditional method. We believe that this was largely due to the fact that the agent received relatively little information at the beginning of the game, and calculating the current strategy in real time increased the errors. To solve this problem, we further improved the agent in the previous second experiment. The experimental results are shown in Table 7.

Table 7 shows that the performance of agent4 based on our WSOS-SG method was better than the performances of HITSZ_HKL and Gibson_3p, winning by 37.4 and 197.4 mbb/h in 20000 hands, respectively. Our agent using the WSOS-SG method took part in the 2018-ACPC and finished in third place for HUNL.

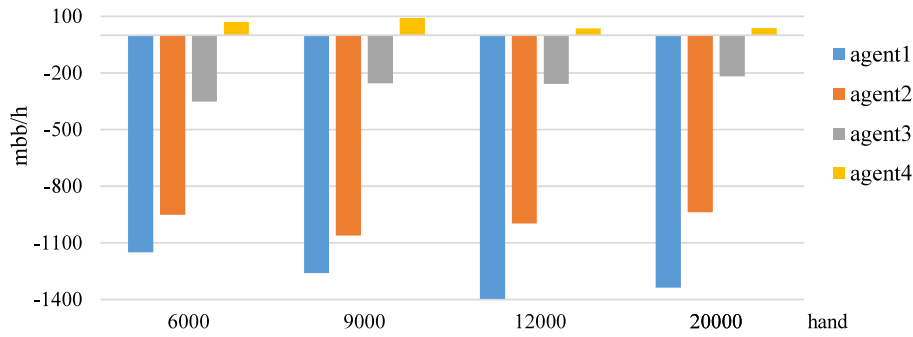


Fig. 3. Performance of agents in HUNL.

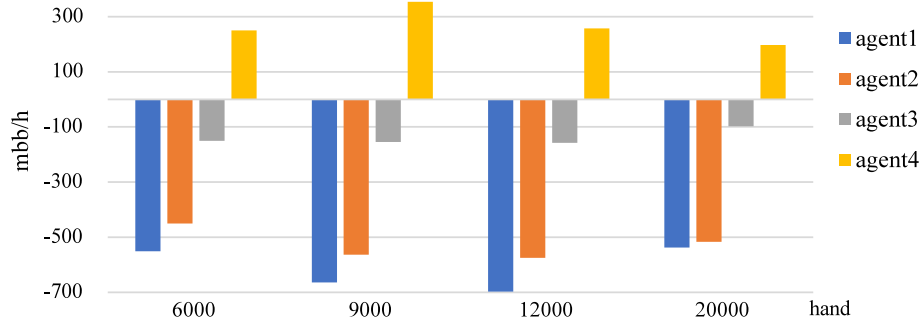


Fig. 4. Performance of agents in TPNL.

Table 8

Performance of our poker agent agent4 in HUNL.

Poker	Agent	Hand			
		6000	9000	12000	20000
HUNL	Agent4 vs. NFSP	109.3	84.1	67.5	76.9
	Agent4 vs. DeepStack1	2.4	-57.1	-15.3	-21.1
	Agent4 vs. Target CFR	179.7	150.1	211.1	190.5
	Agent4 vs. CFR+	27.2	18.4	20.1	19.3
	Agent4 vs. Compact CFR	91.3	102.7	89.2	97.5
	Agent4 vs. Deep CFR	10.1	-17.5	5.4	7.3

Figs. 3 and 4 show the results of the agents based on our method and the traditional method. As shown, agent4 had clear superiority both in the HUNL and TPNL compared to the other agents. Meanwhile, the general trend shows that the experimental results were getting better and better, which indicated that the gradual improvement of the traditional methods was effective from the first experiment to the third experiment. Finally, the performance of the agent with our method, WSOS-SG, was superior to that achieved by the traditional method in the experiment.

As described above, agent4 was based on our method. To better test its performance, we conducted a fourth group experiments with six other high-level agents reported in recent years. The experimental results are shown in Table 8.

Table 8 shows that compared with the improved CFR-based algorithms Target CFR [18], CFR+ [42], and Compact CFR [17], our method had significant advantages, winning by 190.5, 19.3, 97.5 mbb/h in 20000 hands, respectively. This was mainly because our method introduced a real-time intermediate state estimation and blueprint strategy, which made the solution more accurate. Our method outperformed the NFSP [24] and Deep CFR [19], winning 76.9 and 7.3 mbb/h in 20000 hands, respectively. This was because although the NFSP [24] uses a deep neural network for strategy fitting, the optimal strategy requires deep reinforcement learning method to learn. Deep reinforcement learning suffers

from convergence difficulties or significant fluctuations in the process of convergence. As a result, the learned strategy might not reach the theoretical optimum. While the Deep CFR [19] uses neural networks to estimate the regret value, and it reduces the error caused by abstraction. Poker games are multi-round sequential decision games. As the game goes on, more information will be made public. Only using a neural network to estimate the regret value will affect the final result.

The deep neural network is also used in the DeepStack [27], but it uses a neural network to fit the value in each round, which is totally different from the previous two methods that use neural networks. This is a targeted design for poker. The experimental results showed that our method is slightly inferior to DeepStack [27], and our agent, agent4, lost 21.1 mbb/h in 20000 hands. However, our method also has its own advantages. First, the deep neural network in DeepStack is used to estimate the values, which requires a large number of high-quality training samples to train to achieve good results, while our agent does not have this requirement. Second, our agent has good scalability (good performance in both two- and three-player poker, while DeepStack [27] is only suitable for two players).

We also conducted the comparison experiments with several agents in TPNL to further verify the performance of our agent. There are four comparison agents, *Gibson_3p*, *ES - MCCFR*, *NN_3p* and *OppModel*. The agent *ES - MCCFR* is based on ES-MCCFR (external sampling MCCFR), which is also the core algorithm of the six-player agent Pluribus. The agent *NN_3p* is based on neural networks, which used a fully-connected network with seven layers (like DeepCFR, the network takes an information set (observed cards and bet history) as input and outputs probability logits for each possible action). The agent *OppModel* is based on the ES-MCCFR and the opponent model (here the opponent model mainly predicts the distribution probability of the opponent's hand). In the experiment, one player is our agent 4, the other two are both the same comparison agents. Other settings are similar to the setting in the HUNL. The experimental results are as shown in Table 9.

Table 9
Performance of our poker agent agent4 in TPNL.

Poker	Agent	Hand			
		6000	9000	12000	20000
TPNL	Agent4 vs. Gibson_3p	250.7	354.3	257.1	197.4
	Agent4 vs. ES-MCCFR	381	609.2	517.6	490.3
	Agent4 vs. NN_3p	501.2	590	565.1	542.8
	Agent4 vs. OppModel	124	198.1	144	126.5

From Table 9, it can be found that our agent agent4 defeated all four comparison agents in total 20000 hands respectively. It further showed the effectiveness of our agent agent4 in the TPNL. To be specific, our agent winning the agent *Gibson_3p*, *ES-MCCFR*, *NN_3p* and *OppModel* 197.4, 490.3, 542.8 and 126.5 mbb/h in total 20000 hands respectively. This is because for the agent *ES-MCCFR*, limited by the time, it is only trained with the 2000 iterations (in each iteration, the ESMCCFR does 500 traversals). And for the neural network of the agent *NN_3p*, we only refer to the structure of DeepCFR without further detailed adjustments.

In summary, two games were used to test the effectiveness of our method. In general, agent4 based on our method showed excellent performance compared with other methods in comparative experiments. This fully demonstrated the effectiveness of our method. In addition, in the component analysis of our proposed method, the experimental results also verified that our proposed RTHD and ISVE-SG were beneficial to the final solution method.

5. Conclusions and future work

We presented the method WSOS-SG based on the warm-start online solution of sub-game to solve the imperfect-information game. The proposed method has a strong scalability and can be used in both two- and three-player games. In the WSOS-SG, we propose a state value estimation method, which increases the estimation precision of the intermediate state value of the sub-game tree. We also proposed a real-time poker hand abstraction (RTHA) technique, which effectively reduced the error of the abstraction. Our method showed superior performances compared to some state-of-the-art methods in HUNL and TPNL poker.

There are three interesting directions for future work. First, we would like to consider a solution method that requires less expert knowledge. The current method abstracts large-scale games and then solves the abstracted game. This method requires considerable domain knowledge when abstracting. One alternative is to use a neural network to solve the game, like DeepStack [27] and DeepCFR [19]. Second, we would like to develop a method that is more suitable for continuous action spaces. At present, the method for dealing with games with continuous action spaces mainly discretize the continuous action. However, many real games involve continuous action space scenarios. Deep reinforcement learning has shown surprising performance in continuous action space problems. It will be interesting to combine the proposed approach with deep reinforcement learning to solve these types of problems. Third, we would like to study more generalization methods. The success of the method is still limited to specific games, particularly poker games. However, there are many complex problems in reality, and determining how to extend the solution method to such problems is a subject of future work.

CRediT authorship contribution statement

Huale Li: Conceptualization, Methodology, Software, Writing - original draft. **Xuan Wang:** Data curation, Validation, Formal analysis. **Kunchi Li:** Software, Investigation, Methodology.

Fengwei Jia: Visualization, Validation. **Yulin Wu:** Software. **Jiajia Zhang:** Writing - review & editing, Funding acquisition. **Shuhan Qi:** Writing - review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research was funded by key fields R&D project of Guangdong Province, China (No. 2020B0101380001), National Natural Science Foundation of China (No. 61902093), Natural Science Foundation of Guangdong, China (No. 2020A1515010652), Shenzhen Foundational Research Funding, China Under Grant (No. 20200805173048001), PINGAN-HITSz Intelligence Finance Research Center, China, Ricoh-HITSz Joint Research Center, China, GBase-HITSz Joint Research Center, China.

References

- [1] D. Tolpin, S.E. Shimony, MCTS based on simple regret, in: Twenty-Sixth AAAI Conference on Artificial Intelligence, 2012.
- [2] C.B. Browne, E. Powley, D. Whitehouse, S.M. Lucas, P.I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, S. Colton, A survey of Monte Carlo tree search methods, *IEEE Trans. Comput. Intell. AI Games* 4 (1) (2012) 1–43.
- [3] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, *Comput. Sci.* (2013).
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529.
- [5] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, *Nature* 529 (7587) (2016) 484.
- [6] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, et al., Mastering the game of go without human knowledge, *Nature* 550 (7676) (2017) 354.
- [7] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, D. Hassabis, A general reinforcement learning algorithm that masters chess, shogi, and go through self-play, *Science* 362 (6419) (2018) 1140–1144.
- [8] J. Nash, Non-cooperative games, *Ann. of Math.* (1951) 286–295.
- [9] M. Zinkevich, M. Johanson, M. Bowling, C. Piccione, Regret minimization in games with incomplete information, in: *Advances in Neural Information Processing Systems*, 2008, pp. 1729–1736.
- [10] A. Gilpin, T. Sandholm, Finding equilibria in large sequential games of imperfect information, in: *Proceedings 7th ACM Conference on Electronic Commerce*, 2006, pp. 160–169.
- [11] M. Johanson, N. Bard, M. Lanctot, R. Gibson, M. Bowling, Efficient Nash equilibrium approximation through Monte Carlo counterfactual regret minimization, in: *International Conference on Autonomous Agents & Multiagent Systems*, 2012.
- [12] V. Lisy, M. Lanctot, M. Bowling, Online Monte Carlo counterfactual regret minimization for search in imperfect information games, in: *International Conference on Autonomous Agents & Multiagent Systems*, 2015.
- [13] N. Burch, M. Bowling, Cfr-d: Solving imperfect information games using decomposition, 2013, pp. 1–15.
- [14] J. Zhang, H. Liu, Reinforcement learning with monte carlo sampling in imperfect information problems, in: *International Conference on Cognitive Computing*, Springer, 2018, pp. 55–67.
- [15] O. Tammelin, Solving large imperfect information games using cfr+, 2014, *ArXiv Preprint:1407.5042*.
- [16] T.S. Noam Brown, Reduced space and faster convergence in imperfect-information games via pruning, in: *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 596–604.
- [17] E.G. Jackson, Compact CFR, in: *Workshops At the Thirtieth AAAI Conference on Artificial Intelligence*, 2016.
- [18] E.G. Jackson, Targeted CFR, in: *Workshops At the Thirty-First AAAI Conference on Artificial Intelligence*, 2017.

- [19] N. Brown, A. Lerer, S. Gross, T. Sandholm, Deep counterfactual regret minimization, in: *International Conference on Machine Learning*, 2019.
- [20] D. Billings, A. Davidson, J. Schaeffer, D. Szafron, The challenge of poker, *Artificial Intelligence* 134 (1) (2002) 201–240.
- [21] D. Billings, N. Burch, A. Davidson, R.C. Holte, J. Schaeffer, T. Schauenberg, D. Szafron, Approximating game-theoretic optimal strategies for full-scale poker, in: *International Joint Conference on Artificial Intelligence*, 2003.
- [22] K. Hu, S. Ganzfried, Midgame solving: A new weapon for efficient large-scale equilibrium approximation, in: *29th International Conference on Tools with Artificial Intelligence*, 2017.
- [23] R. Gibson, D. Szafron, Regret minimization in multiplayer extensive games, in: *Twenty-Second International Joint Conference on Artificial Intelligence*, 2011.
- [24] J. Zhang, H. Liu, Reinforcement learning with Monte Carlo sampling in imperfect information problems, in: *International Conference on Cognitive Computing*, 2018.
- [25] J. Shi, M.L. Littman, Abstraction methods for game theoretic poker, in: *International Conference on Computers and Games*, Springer, 2000, pp. 333–345.
- [26] A. Gilpin, T. Sandholm, A competitive Texas Hold'em poker player via automated abstraction and real-time equilibrium computation, in: *International Joint Conference on Autonomous Agents & Multiagent Systems*, 2006.
- [27] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, M. Bowling, Deepstack: Expert-level artificial intelligence in heads-up no-limit poker, *Science* 356 (6337) (2017) 508–513.
- [28] N. Brown, T. Sandholm, Safe and nested subgame solving for imperfect-information games, in: *Advances in Neural Information Processing Systems*, 2017, pp. 689–699.
- [29] N. Brown, T. Sandholm, Superhuman ai for heads-up no-limit poker: Libratus beats top professionals, *Science* 359 (6374) (2017) 1733.
- [30] N. Brown, T. Sandholm, Superhuman ai for multiplayer poker, *Science* 365 (6456) (2019) 885–890.
- [31] R.G. Gibson, Regret minimization in games and the development of champion multiplayer computer poker-playing agents, 2014.
- [32] R. Cressman, C. Ansell, K. Binmore, *Evolutionary Dynamics and Extensive Form Games*, vol. 5, MIT Press, 2003.
- [33] L.K. Simon, M.B. Stinchcombe, Extensive form games in continuous time: Pure strategies, *Econometrica* (1989) 1171–1214.
- [34] N. Brown, T. Sandholm, Solving imperfect-information games via discounted regret minimization, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33, pp. 1829–1836.
- [35] J. Zhang, L. Hong, Building endgame data set to improve opponent modeling approach, in: *IEEE Second International Conference on Data Science in Cyberspace*, 2017.
- [36] R. Gibson, Regret minimization in non-zero-sum games with applications to building champion multiplayer computer poker agents, 2013, arXiv preprint [arXiv:1305.0034](https://arxiv.org/abs/1305.0034).
- [37] K. Waugh, A fast and optimal hand isomorphism algorithm, in: *Workshops At the Twenty-Seventh AAAI Conference on Artificial Intelligence*, 2013.
- [38] A. Likas, N. Vlassis, J.J. Verbeek, The global k-means clustering algorithm, *Pattern Recognit.* 36 (2) (2003) 451–461.
- [39] T. Denoeux, Analysis of evidence-theoretic decision rules for pattern classification, *Pattern Recognit.* 30 (7) (1997) 1095–1107.
- [40] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks, in: *International Conference on Neural Information Processing Systems*, 2012.
- [41] L. Kocsis, C. Szepesvári, Bandit based monte-carlo planning, in: *European Conference on Machine Learning*, Springer, 2006, pp. 282–293.
- [42] N. Burch, M. Moravcik, M. Schmid, Revisiting CFR+ and alternating updates, *Artificial Intelligence* 64 (2019) 429–443.